



AULA 1 UNIDADE 5

Controle de Versão



Introdução

- Como é possível controlar todo um processo de desenvolvimento de *software* sem medo de perder informações ou sobrescrever atualizações?
 - Afinal, essa habilidade é necessária para o Fluxo de Valor de TI.
 - A solução são Sistemas de Controle de Versão (do inglês, *Source Code Management* – SCM).
- Possibilidades que um SCM oferece:
 - Acompanhar históricos de desenvolvimento.
 - Customização de uma determinada versão.
 - Recuperar versões anteriores.

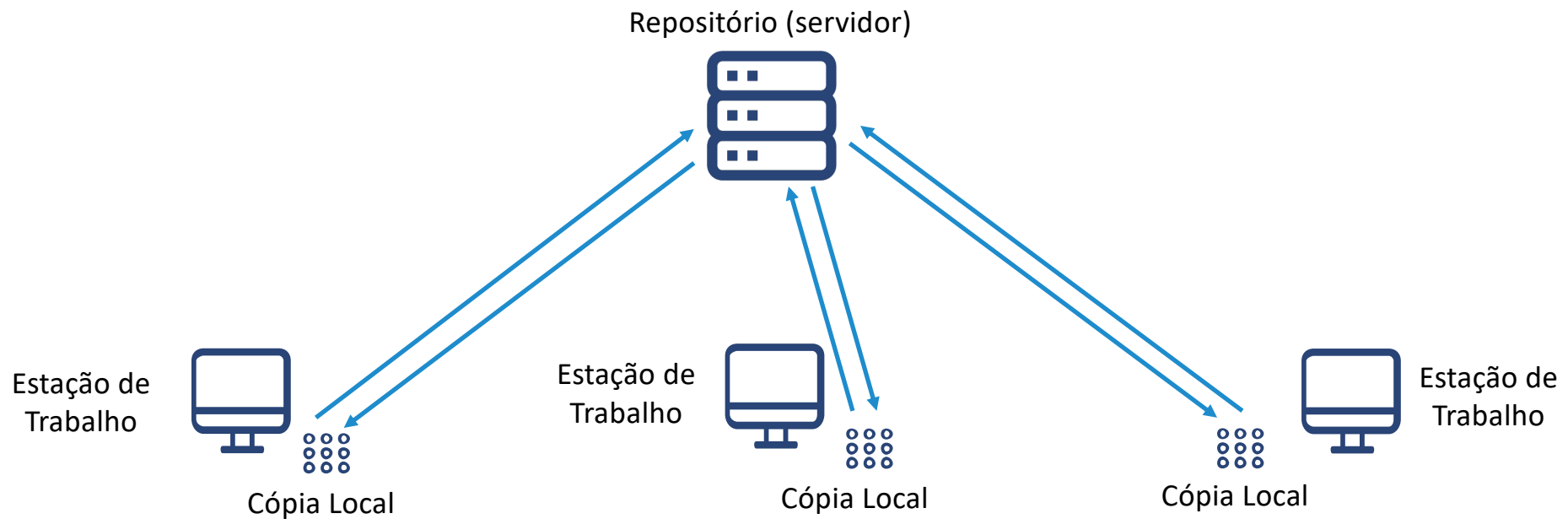


Controle de Versão

- Os arquivos de um projeto são armazenados em um **repositório** (em um servidor):
 - Código fonte propriamente dito.
 - Estruturas de dados otimizadas que registram as versões do código (**árvore de revisões**).
- Desenvolvedores interagem com a **versão cliente** da ferramenta (nas suas estações de trabalho):
 - Atuam diretamente na sua cópia local.
 - Submetem as alterações ao repositório, criando uma **revisão**.
 - Recuperam as alterações de outros desenvolvedores.
- Registrando o histórico:
 - Toda alteração é registrada: autor, data e origem das alterações.
 - É possível desfazer alterações.
- Colaboração:
 - Vários desenvolvedores podem fazer alterações em paralelo.
 - Se atuam em trechos diferentes de código, não há conflito. Caso contrário, o sistema alerta para as diferenças.



Controle de Versão



Sistemas de Controle de Versão

- Git:
 - Protocolo criado por Linus Torvalds para o *kernel* do Linux.
 - Foco na eficiência e rapidez.
 - Arquitetura descentralizada.
 - Implementado pelo GitHub e o *BitBucket*.
- Mercurial:
 - Assim como o Git, prioriza a descentralização.
 - Tem menos comandos, sendo mais simples do que o Git.
- Subversion:
 - Protocolo centralizado.
 - Não existe a cópia local divergente, todas mudanças são aplicadas diretamente no repositório.



Operações em Controle de Versões

- Operações mais comuns:
 - **Commit**: criação de nova versão do projeto.
 - **Checkout**: recuperação de uma versão específica do projeto.
 - **Revert**: descartar mudanças locais, recuperando uma versão do repositório.
 - **Diff**: comparar uma versão do arquivo na estação local com qualquer outra versão do repositório.
 - **Delete**: exclusão de arquivos do repositórios.
 - **Lock**: travamento do arquivo.
- Apesar de utilizarem nomes diferentes, todos protocolos suportam essas operações.

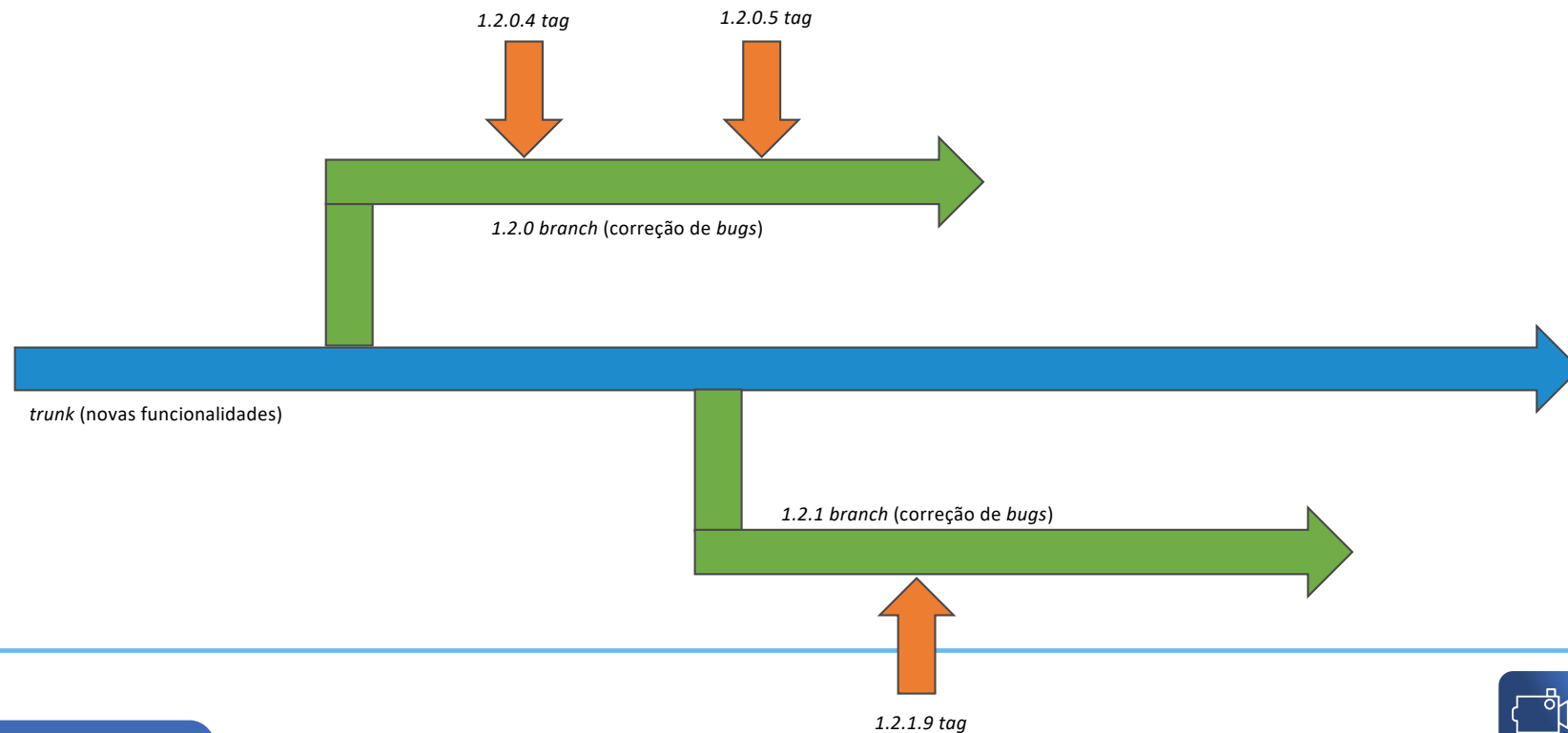


Operações em Controle de Versões

- Uma característica importante dos sistemas de controle de versões é a possibilidade de separar modificações em um ramo (*branch*) diferente.
- Um *branch* pode ser usado para implementar novas funcionalidades sem comprometer o caminho principal da implementação (*trunk*).
- O *branch* só pode ser integrado ao *trunk* se estiver estável.
- Uma revisão pode ser rotulada (*tag*) como uma versão fixa com funcionalidades estáveis que não sofrerão alterações.



Operações em Controle de Versões



Centralizado versus Descentralizado

- Repositório Centralizado:
 - Único servidor central, arquitetura cliente-servidor.
 - A cópia local de cada estação de trabalho é idêntica.
 - Os desenvolvedores realizam todas as operações com efeito imediato no repositório do servidor.
- Repositório Descentralizado (ou distribuído):
 - A cópia local é um **repositório local**.
 - O desenvolvedor pode criar várias versões locais e só depois submetê-las.
 - As estações de trabalho poderia trocar versões entre si, mas recomenda-se um servidor central que atua como sincronizador entre as cópias locais.



Centralizado versus Descentralizado

Descrição	Centralizado	Distribuído
Criação de cópia local	CHECKOUT	CLONE
Envio de modificações	COMMIT	COMMIT
Alteração da cópia local para uma revisão	UPDATE	UPDATE
Importação de revisões feitas em outro repositório		PULL
Envio de revisões locais para outro repositório		PUSH



Conclusão

- Repositórios centralizados são mais simples, porém como todas as alterações são submetidas imediatamente ao servidor, o mesmo torna-se um gargalo.
- Repositórios distribuídos são mais complexos, pois o desenvolvedor precisa lidar com a cópia local como um repositório. Entretanto, como a frequência de submissões ao repositório no servidor é menor, temos maior escalabilidade.
- O protocolo com maior sucesso é o sistema descentralizado Git.
- A criação de *branches* e o uso de *tags* se encaixam na definição do fluxo de valor de TI, pois permitem experimentação e equipes trabalhando de forma independente.





GRADUAÇÃO ONLINE FGV

TECNOLÓGICA